# Lab Ass. 1: Image classification. CNN architectures: Transfer Learning and Image Data Augmentation

This lab assignment is implemented in Google Colab, an environment that allows you to develop Python code in Python notebooks and use GPU acceleration to train your Deep Learning models. To use it you will need to create a personal Google account. To work in Google Colab, follow these instructions:

- Create a folder for the assignments of the course in your personal Google Drive.
- Open the Colab notebook links included in this lab script
- Save a copy of the notebooks in your Google Drive folder
- Each Colab notebook (i.e. files ending in .ipynb) corresponds to a part of the assignment question. In Google Drive, double click on the notebook, and select `open with Colab`.
- Once you have completed the assignment (i.e. reached the end of the notebook), you can save your edited file back to your Drive and move on to the next notebook.
- Ensure you are periodically saving your notebook (File → Save), so that you don't lose your progress if you step away from the assignment and the Colab VM disconnects.

The objective of this lab assignment is to present the student with the image classification task, the basics of several CNN architectures, handling image datasets with PyTorch; and transfer learning and image data augmentation techniques in the context an image classification task. To complete this assignment, you will need to read the Pytorch documentation. You can find it here.

You need to complete the code in the Colab notebooks and a lab report with the answers to the questions that are included in the following Sections. Once you have finished it, you must upload in Moodle a copy of the completed notebooks (.ipynb files) and the lab report, combined in a single zip file. **IMPORTANT: Both the report file and the zip file must be named in the following format: 'SURNAME(s)_NAME'**

You can find here the first Colab notebook. You must complete the code in the notebook and answer the questions in Section 1.1.

## *1.1 Simple CNN*

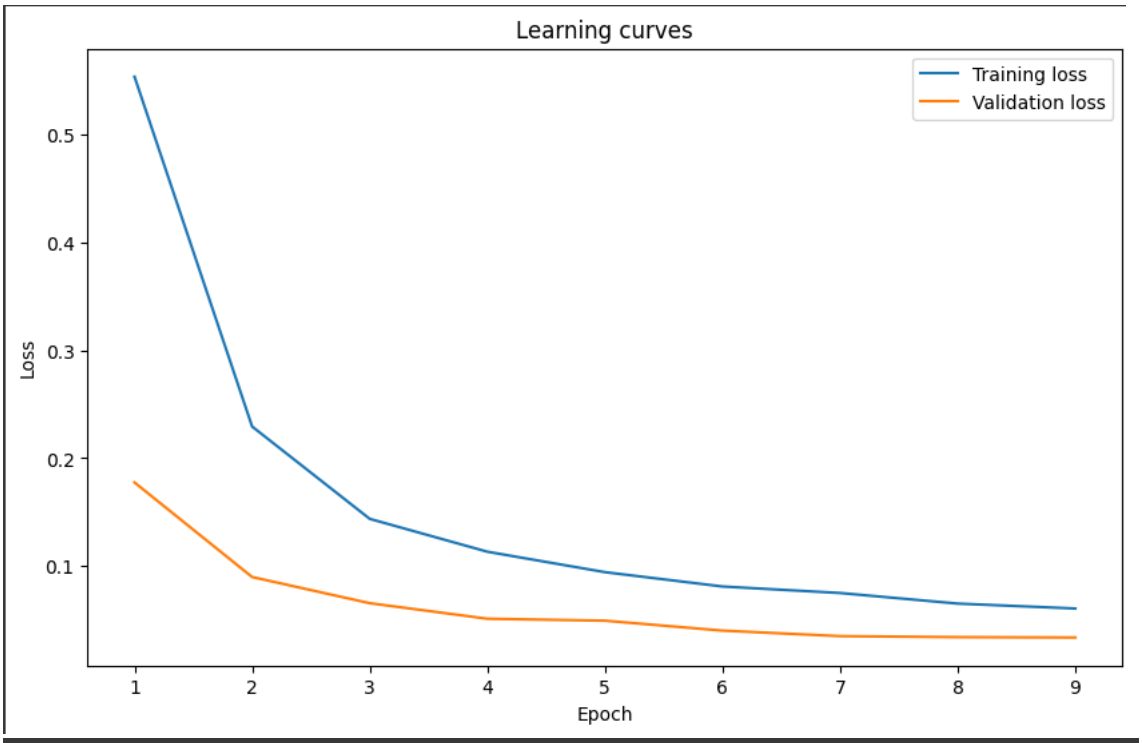
- Report the sizes of the training and validation sets downloaded from the MNIST dataset

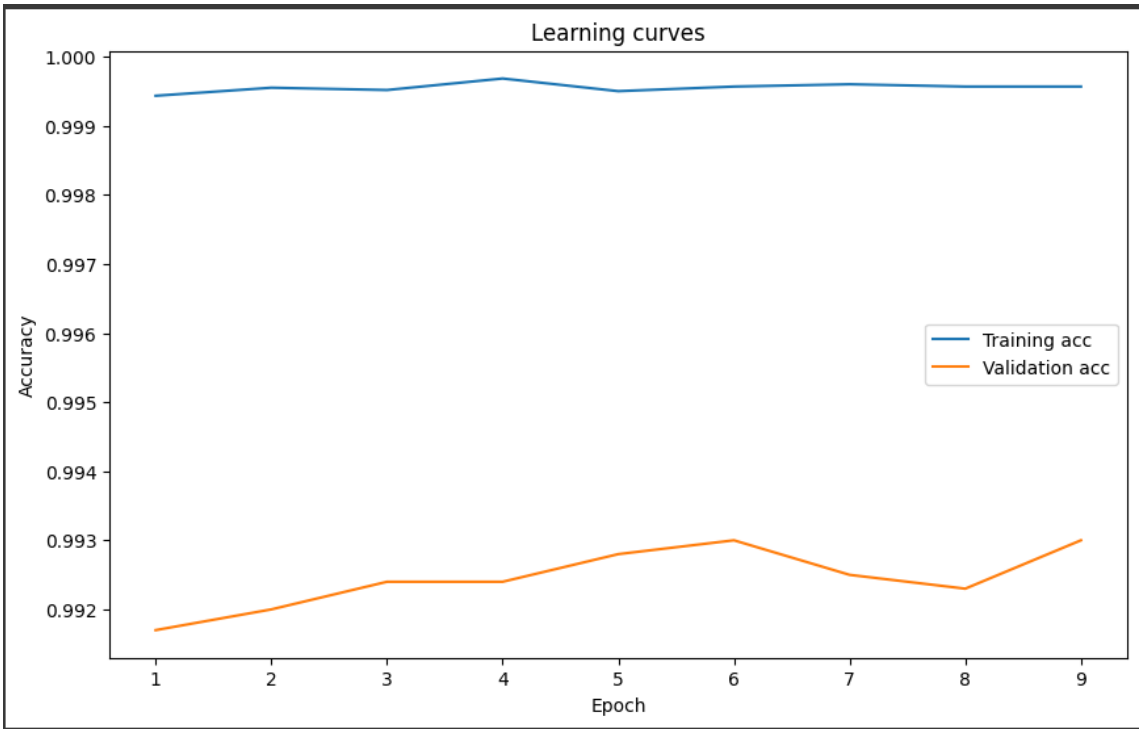
| | Image height | Image width | # image channels | # samples |
|---|---|---|---|---|
| Training | 28 | 28 | 1 | 60,000 |
| Validation | 28 | 28 | 1 | 10,000 |

- Report the number of parameters of the SimpleCNN model

| | # trainable parameters |
|---|---|
| Simple CNN | 813,802 |

- Include the training and validation curves for 10 epochs. Also indicate the best accuracy result and in which epoch is this achieved



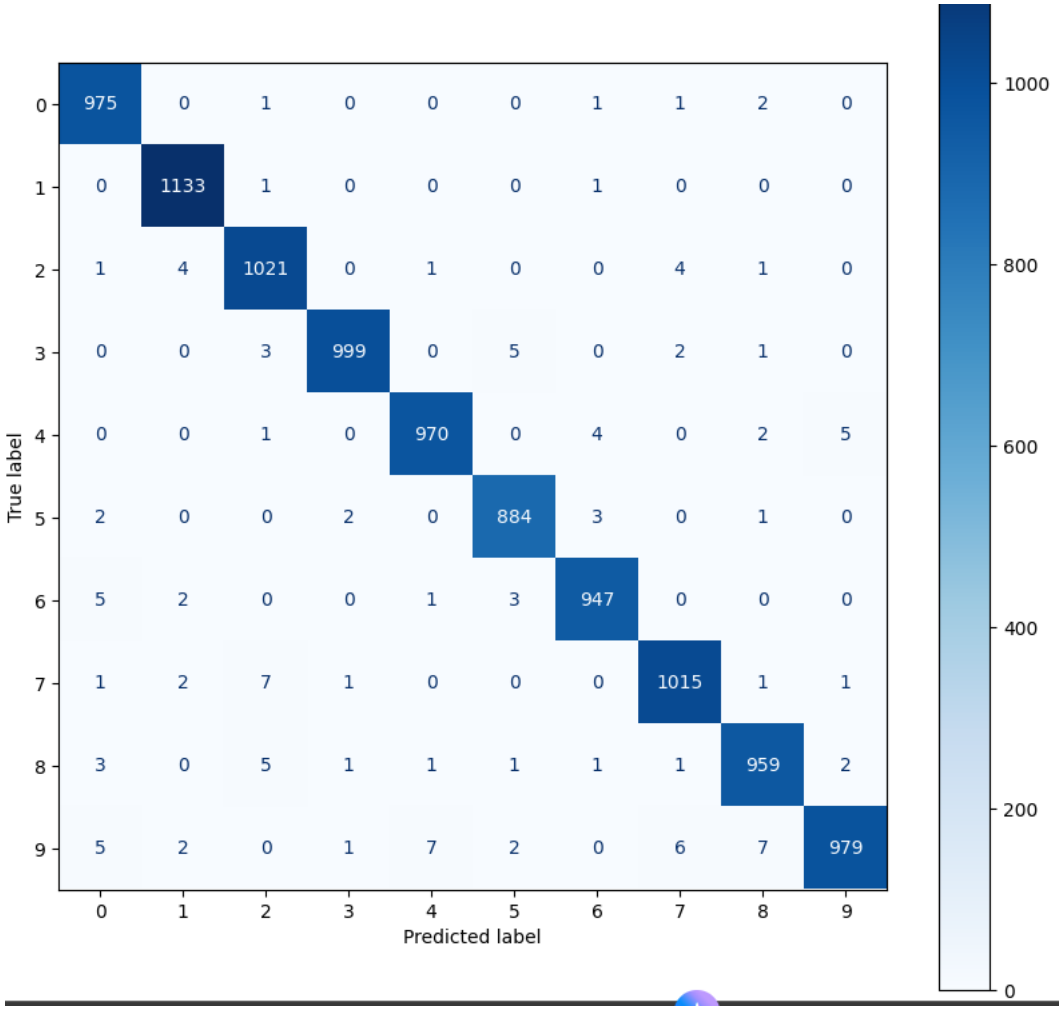




As shown in the training and validation curves, the loss consistently decreases throughout the epochs, while the accuracy stabilizes near the end of the training. This behavior is expected, since the loss function is a more sensitive metric than accuracy — once accuracy saturates, the model may still continue to refine its internal representations. A slight overfitting tendency can be observed in both curves, but it remains moderate and could be mitigated through regularization techniques such as dropout or weight decay. Given the simplicity of the MNIST dataset, the high accuracy achieved is reasonable and not indicative of an error. Since both the training and validation metrics are stable, additional epochs would likely yield little improvement. However, introducing regularization could potentially enhance generalization slightly, even though the best performance has already been reached by the last epoch.

| | Best accuracy (validation) | Epoch with best accuracy |
|---|---|---|
| Simple CNN | 99.32% | 9 |

Comment on the conclusions of the training and validation losses, regarding possible high-bias or overfitting problems. Indicate if you consider further training would improve the performance of the model

- Include the confusion matrix. Given this confusion matrix, report the 2 cases of confusion between classes that happen more frequently.

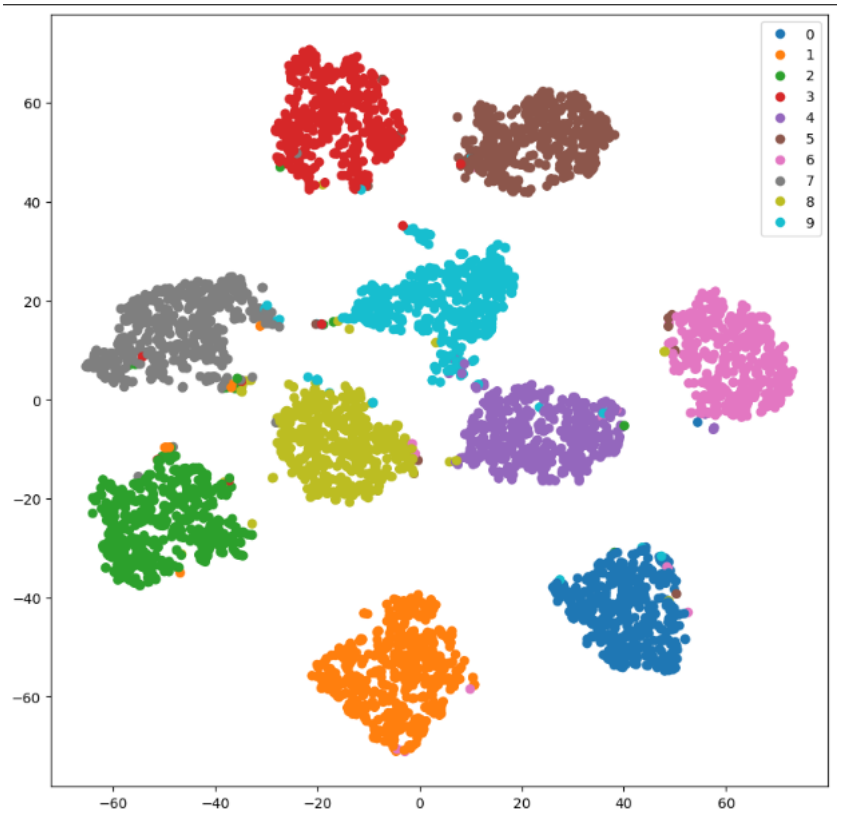



The model most frequently confuses the digits 8 and 9, 4 and 9, and 2 and 7. These errors are intuitive because these digit pairs share visually similar shapes and are commonly mistaken even by humans. To further improve performance, targeted data augmentation with more examples of these ambiguous digits or introducing new data of these categories could help, although care must be taken to avoid class imbalance. Despite these confusions, the overall accuracy is already very high, suggesting that the model's performance is close to human-level recognition for this dataset.

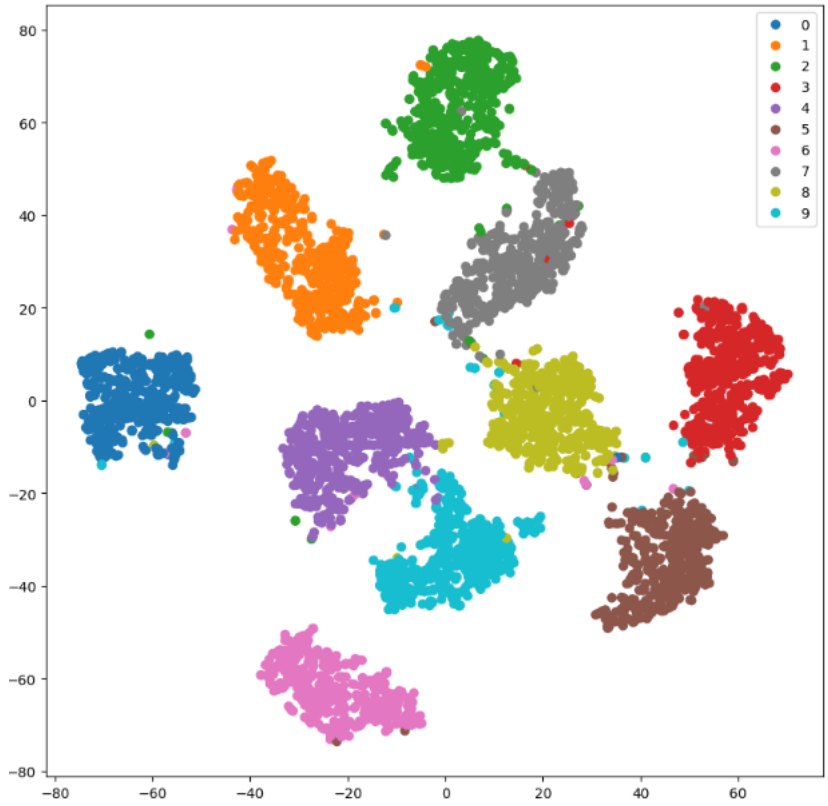
- Comment on the differences between the t-SNE graph of the representation of the final and intermediate layers of the CNN, applied to the images of the validation set. Consider the

proximity and the dispersion of the clusters in both representations.

Intermidiate representations:




Final representations:




The t-SNE visualizations of the intermediate and final layers show reasonably well-separated clusters, although some outliers are still present. The final layer exhibits slightly tighter groupings, indicating that the model progressively refines its feature representations. This suggests that high-level discriminative features are effectively captured in the middle layers, and reducing the network's depth could lower computational cost with only a small penalty in classification accuracy.

- Given the differences between the t-SNE representation of both layers, and using the architecture definition, identify at which network level are the features extracted, and propose a way to reduce the network complexity with only a low penalty in classification accuracy.

As mentioned previously reducing the amount of layers eliminating the last ones could lead into a better performance

You can find here the second Colab notebook. You must complete the code in the notebook and answer the questions in Sections 1.2 and 1.3.

## *1.2 AlexNet*

- Include the code you have used to define the `class Alexnet`:

```
class AlexNet(nn.Module):
    def __init__(self, output_dim):
        super().__init__()

        self.features = nn.Sequential(
            # First convolutional layer. Use 5x5 kernel
instead of 11x11
```

```
            nn.Conv2d(3, 48, 5, 2, 2), #in_channels,
out_channels, kernel_size, stride, padding
            nn.MaxPool2d(2), #kernel_size
            nn.ReLU(inplace = True),
            # Complete the following four conv layers of
the AlexNet model.
            # Subsampling is only performed by 2x2 max
pooling layers (not with stride in the
            # convolutional layers)
            # Pay special attention to the number of input
and output channels of each layer
            nn.Conv2d(48,128,5,1,2),
            nn.ReLU(inplace = True),
            nn.MaxPool2d(2),

            nn.Conv2d(128,192,3,1,1),
            nn.ReLU(inplace = True),

            nn.Conv2d(192,192,3,1,1),
            nn.ReLU(inplace = True),

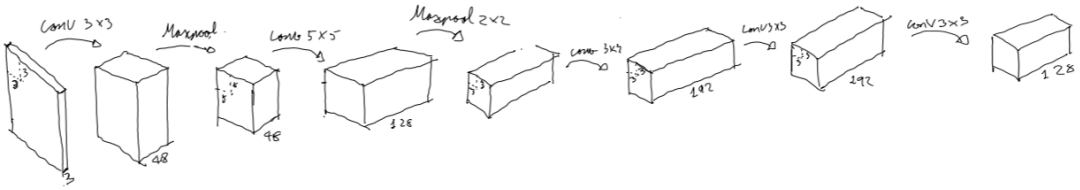
            nn.Conv2d(192,128,3,1,1),
            nn.ReLU(inplace = True),
            nn.MaxPool2d(2)

        )

        self.classifier = nn.Sequential(
            # First linear layer
            nn.Dropout(0.5),
            nn.Linear(128 * 2 * 2, 2048), # final conv
layer resolution 2x2
            nn.ReLU(inplace = True),
            # second linear layer
            nn.Dropout(0.5),
            nn.Linear(2048, 2048),
            nn.ReLU(inplace = True),
            nn.Linear(2048, output_dim),
        )

    def forward(self, x):
        x = self.features(x)
        interm_features = x.view(x.shape[0], -1)
        x = self.classifier(interm_features)
        return x, interm_features
```

Following the instructions of the code and the architecture draw this architecture implementation is presented as our solution. The check was correct so the architecture can be considered correct.



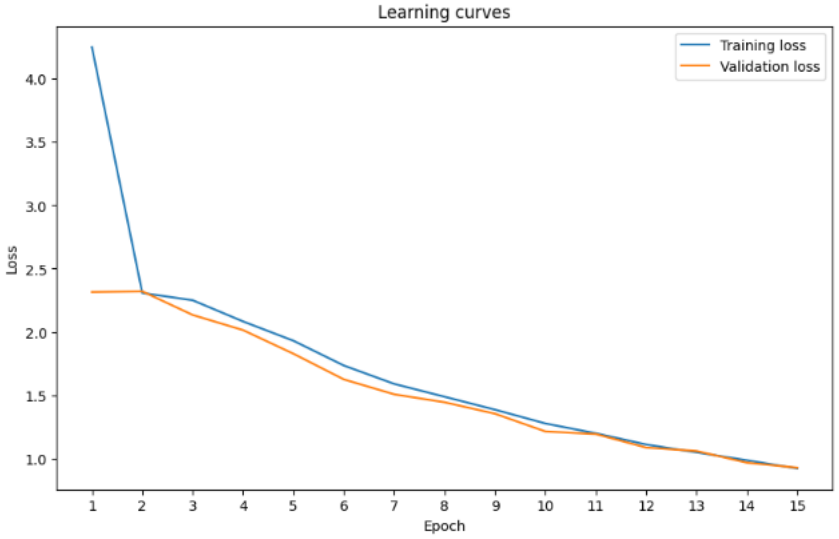

- Report the number of parameters of the AlexNet model

| | # trainable parameters |
|---|---|
| AlexNet | 8,228,008 |

- Include the training and validation curves for 15 epochs. Also indicate the best accuracy result and in which epoch is this achieved

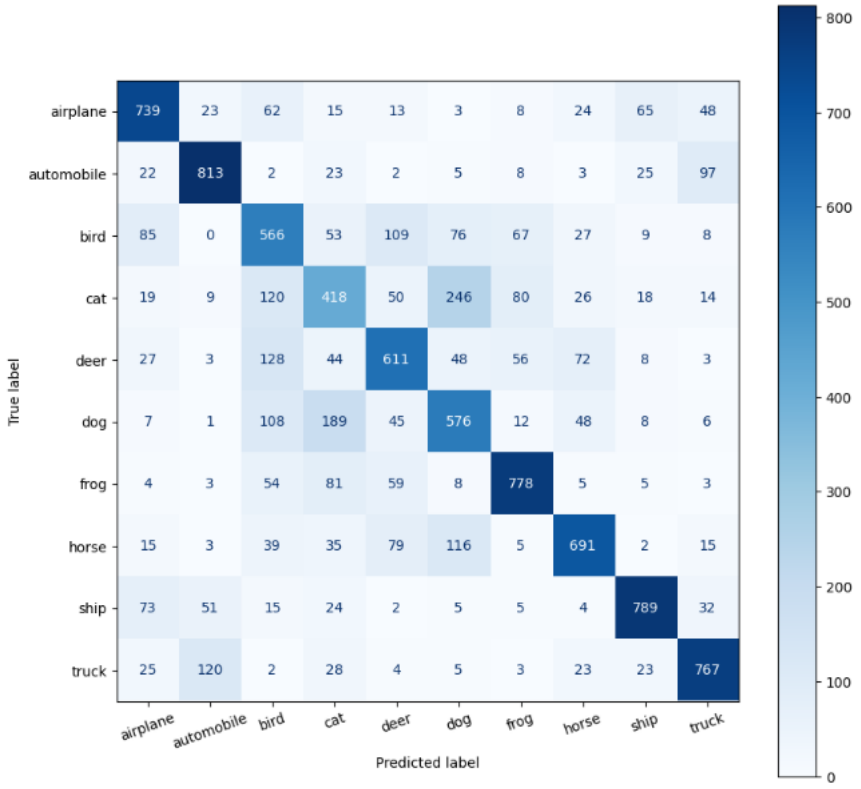
| | Best accuracy (validation) | Epoch with best accuracy |
|---|---|---|
| AlexNet | 67.48% | 15 |

Comment on the conclusions of the training and validation losses, and remark what is possibly happening after epoch 10. Indicate if you consider further training would improve the performance of the model



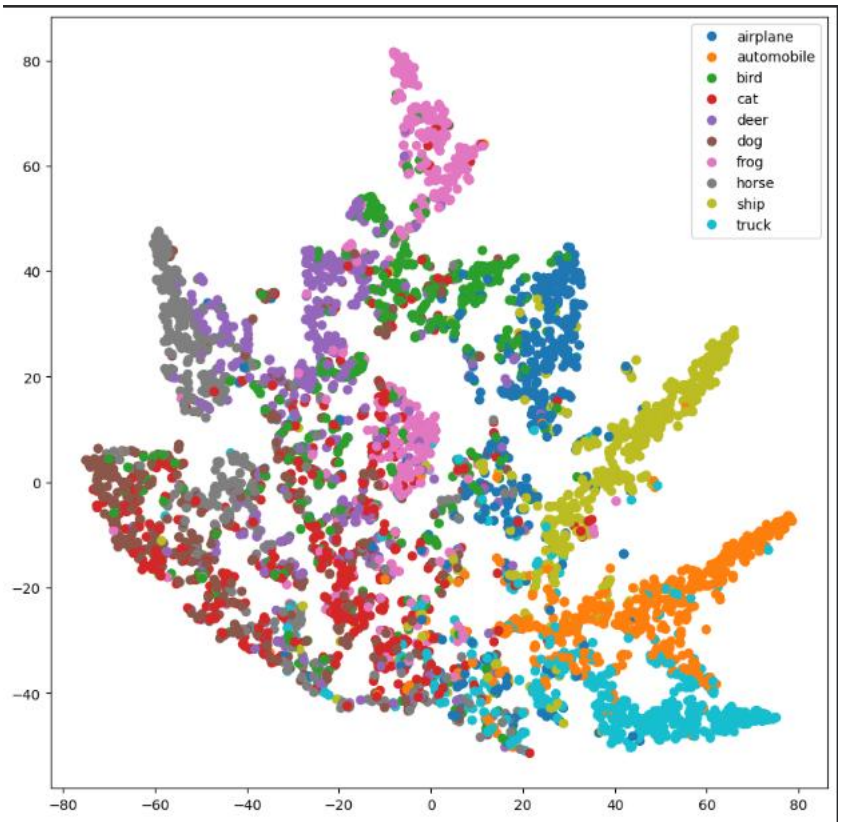

Regarding training performance, the accuracy and loss curves show that the model continues to improve throughout the 15 epochs, with the best validation accuracy achieved at the final epoch. This suggests that the network has not yet converged, and extending the training for additional epochs could further improve performance. The absence of clear overfitting indicates that the model is still underfitting to some extent, which is understandable considering AlexNet's higher complexity and parameter count compared to the previous Simple CNN (over ten times larger). Consequently, longer training or fine-tuning hyperparameters such as learning rate and batch size could lead to significantly better results.

- Include the confusion matrix. Comment on the results attending to the characteristics of the image classes




The confusion matrix reveals that cats and dogs are often misclassified as each other, which is unsurprising due to their similar textures and poses. Additionally, birds are sometimes confused with deer, cats, or dogs, indicating that the model struggles with recognizing avian features, possibly due to the variability in their shapes and backgrounds. Including more representative samples of birds or applying stronger data augmentation could help mitigate these issues by improving robustness to intra-class variability.

- Include the t-SNE results for the final layer: discuss these results (proximity, dispersion, cluster aggrupation) considering the appearance of the different classes, their typical features, and compare the results with the t-SNE results in the MNIST dataset.




The t-SNE visualization of the AlexNet's final layer shows less distinct and more overlapping clusters compared to the MNIST dataset results, reflecting the higher complexity and diversity of the CIFAR-like image dataset. While some grouping by class is visible—particularly along the cluster edges—the separability is less pronounced. This suggests that the model's learned features are less linearly separable, likely due to the more challenging nature of natural image classification tasks.

## *1.3 Basic Residual Network*

- Include the code you have used to define the `class BasicResBlock` and `class BasicResNet`

```
class BasicResBlock(nn.Module):
    def __init__(self, in_channels, out_channels,
kernel_size, stride=1, padding=1, downsample=False):
        super().__init__()

        self.conv1 = nn.Conv2d(in_channels, out_channels,
kernel_size,
                               stride=stride,
padding=padding, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        self.conv2 = nn.Conv2d(out_channels, out_channels,
kernel_size,
                               stride=1, padding=padding,
bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

```

```
            if downsample:
                conv1D = nn.Conv2d(in_channels, out_channels,
    kernel_size=1,
                                   stride=stride, bias=False)
                bn = nn.BatchNorm2d(out_channels)
                self.downsample = nn.Sequential(conv1D, bn)
            else:
                self.downsample = None

        def forward(self, x):
            identity = x

            out = self.conv1(x)
            out = self.bn1(out)
            out = self.relu(out)
            out = self.conv2(out)
            out = self.bn2(out)

            if self.downsample is not None:
                identity = self.downsample(identity)

            out += identity
            out = self.relu(out)

            return out

    class BasicResNet(nn.Module):
        def __init__(self, output_dim):
            super().__init__()

            # define the 8 convolutional layers using the
    BasicResBlock (4 blocks)
            self.layer1= BasicResBlock(3, 32, 3, stride = 2 ,
    downsample = True)
            self.layer2= BasicResBlock(32, 32, 3, stride =
    1,  downsample = False)
            self.layer3= BasicResBlock(32,64, 3, stride = 2 ,
    downsample = True)
            self.layer4= BasicResBlock(64, 64, 3, stride =
    1,  downsample = False)
            # define an adaptive average pooling layer with an
    output size of 2x2
            # use nn.AdaptiveAvgPool2d
            self.avgpool= nn.AdaptiveAvgPool2d((2,2))

            # classifier with 1 layer
            self.classifier = nn.Sequential(
                # First linear layer
                nn.Dropout(0.5),
```

```
            nn.Linear(64 * 2 * 2, output_dim),
        )

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.avgpool(x)

        interm_features = x.view(x.shape[0], -1)
        x = self.classifier(interm_features)
        return x, interm_features
```

- Report the number of parameters of the BasicResNet model

| | # trainable parameters |
|---|---|
| BasicResNet | 163,210 |

- Include the training and validation curves for 15 epochs. Also indicate the best accuracy result and in which epoch is this achieved

| | Best accuracy (validation) | Epoch with best accuracy |
|---|---|---|
| BasicResNet | 75,21% | 13 |

- Comment on the trade-off between computational complexity (memory) and accuracy in the implemented AlexNet and ResNet models. Compare this trade-off in both architectures to achieve a conclusion on the efficiency of these two different architectures.

As expected, both architectures show a remarkable difference in terms of computational requirements and classification performance. The AlexNet model contains 8,228,008 trainable parameters, whereas the BasicResNet only has 163,210, making the latter approximately 80 times less resource-demanding. Despite this drastic reduction in parameters, BasicResNet achieves a higher accuracy of 75.21%, compared to AlexNet's 67.48%, both measured at their respective best epochs.

These results are not coincidental but rather reflect the evolution of deep learning architectures following the ImageNet competition breakthroughs. Starting with AlexNet, which introduced deeper convolutional networks for large-scale image classification, subsequent architectures such as GoogLeNet optimized efficiency through 1×1 convolutions, drastically reducing computational costs. Finally, ResNet introduced skip (residual) connections, effectively addressing the vanishing gradient problem and allowing deeper,

more stable networks. These design innovations collectively led to superior performance with lower computational overhead, demonstrating a much better trade-off between accuracy and efficiency. In summary, BasicResNet represents a significant architectural improvement, achieving higher accuracy while using only a fraction of the computational resources required by AlexNet.

You can find here the third Colab notebook. You must complete the code in the notebook and answer the questions in Section 1.4

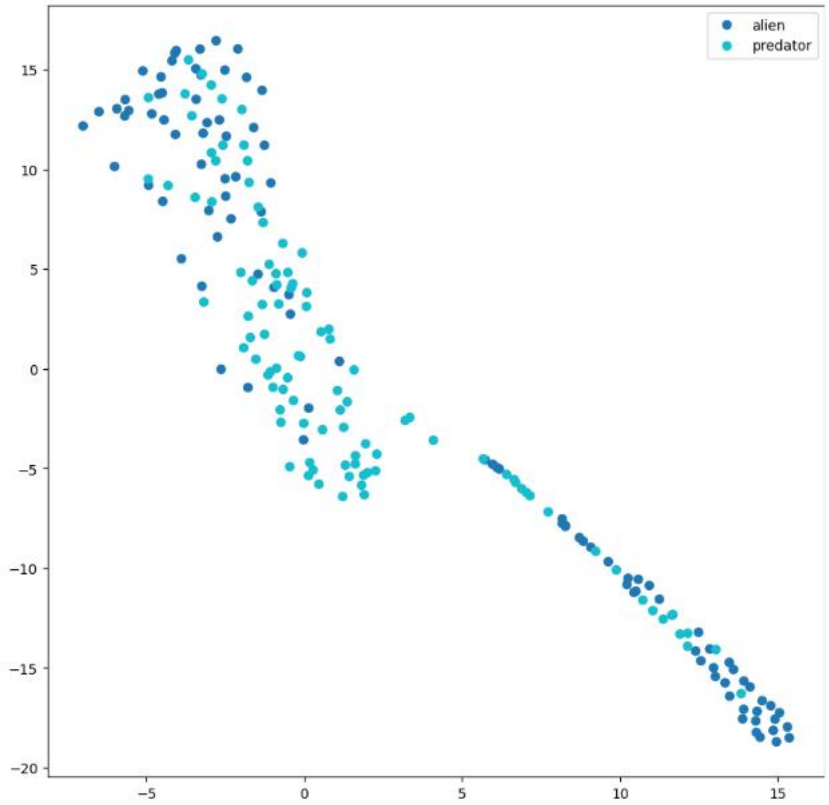
## *1.4 Transfer Learning*

- Report the accuracies obtained for the different alternatives:

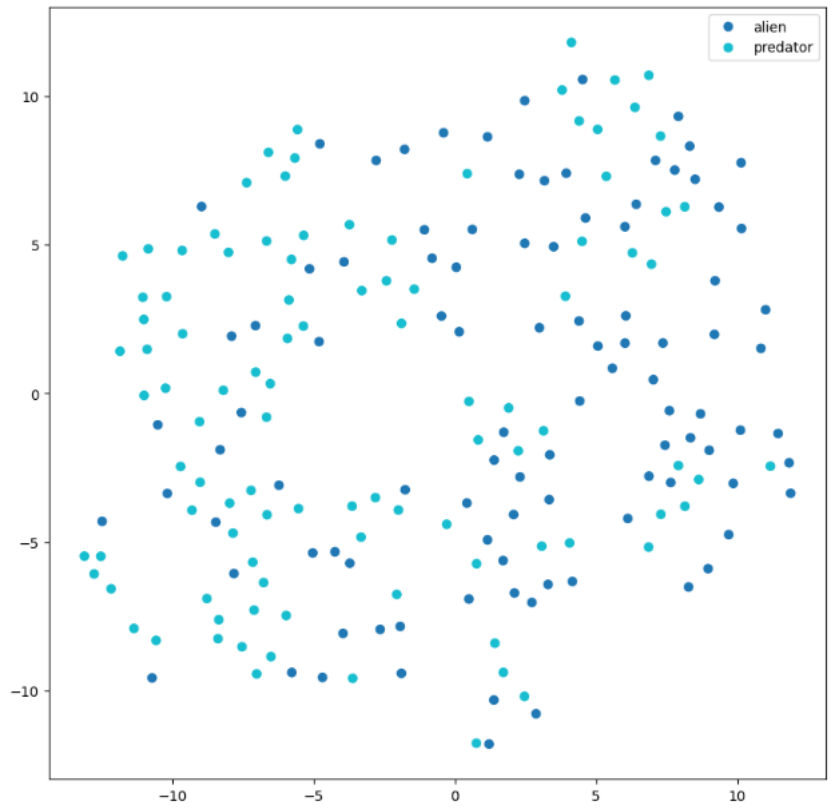
| | Trained from scratch | Pre-trained + SVM | Fine tuning (w/o data augmentation) | Fine tuning (w data augmentation) |
|---|---|---|---|---|
| Accuracy | 75.5% | 93.5% | 94.5% | 95% |

The results clearly demonstrate the benefits of transfer learning. The accuracy improves substantially when using pre-trained models compared to training from scratch (75.5% → 93.5%), showing that leveraging pre-learned feature representations greatly enhances classification performance. Fine-tuning the model further increases accuracy (94.5%), with a slight additional gain when incorporating data augmentation (95%). These results confirm that data augmentation provides better generalization by exposing the model to more diverse input variations. Although the improvements between fine-tuning and fine-tuning are smaller this is probably because the model already got an unexpected high accueracy of 93.5% when it was supposed to reach an accuracy of around 90%.

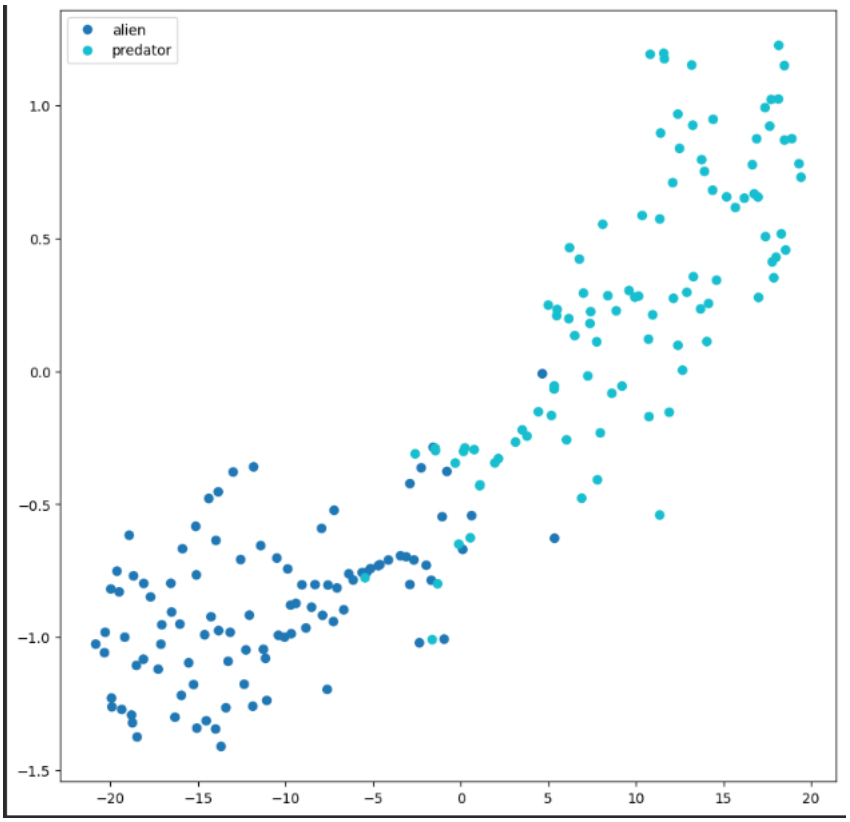
- Include in the report the t-SNE representations of the different alternatives: trained from scratch, pre-trained, fine-tuned (without data augmentation), and fine-tuned (with data augmentation).
  Trained from scratch:



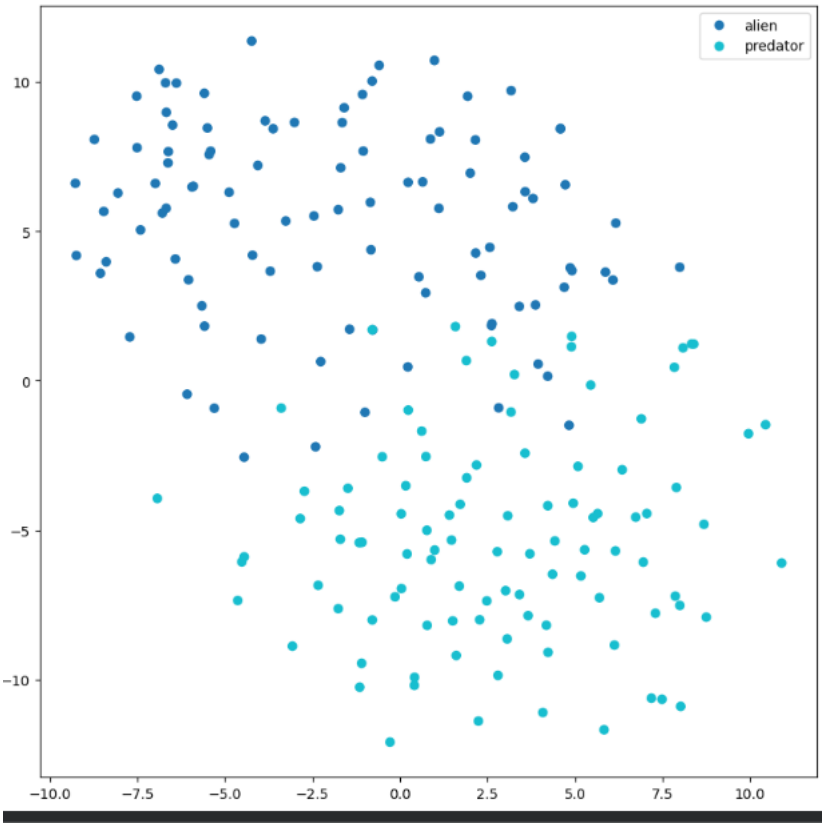

Pretrained features:




Fine tunning without data augmentation:




Fine tunning with data augmentation:




The t-SNE representations show that models trained from scratch produce more dispersed and overlapping clusters, indicating weaker feature separability. In contrast, pre-trained and fine-tuned models generate more compact and clearly separated clusters, reflecting stronger discriminative feature learning. The fine-tuned models—particularly those trained with data augmentation—display the best separation, with

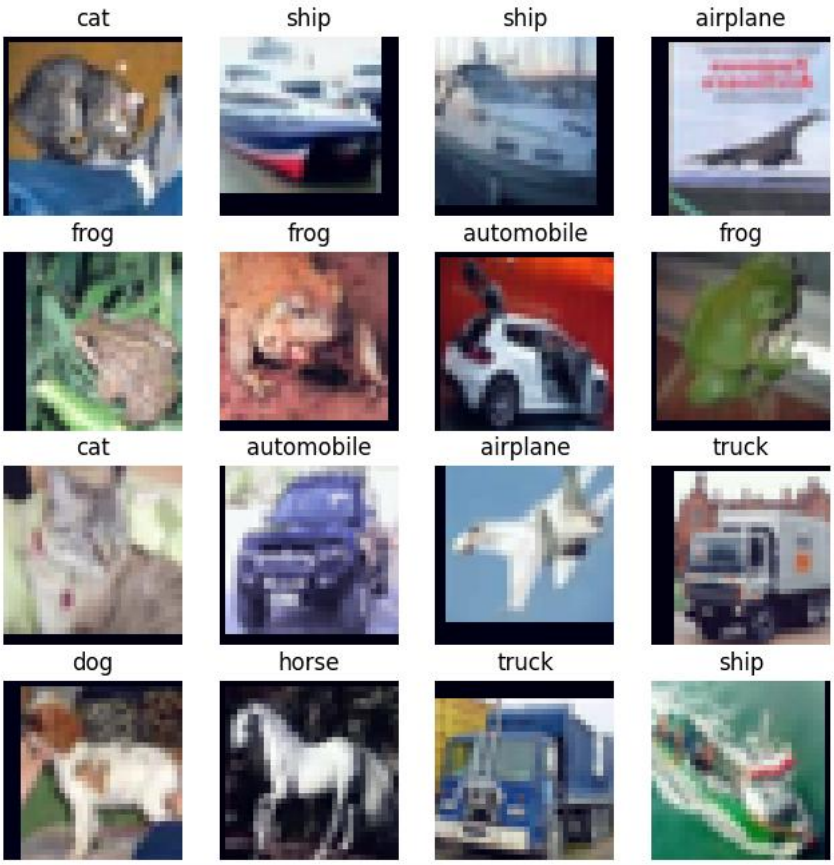
minimal overlap and fewer misclassified samples. This confirms that fine-tuning on pre-trained representations significantly enhances linear separability and classification confidence.

You can find here the fourth Colab notebook. You must complete the code in the notebook and answer the questions in Section 1.5

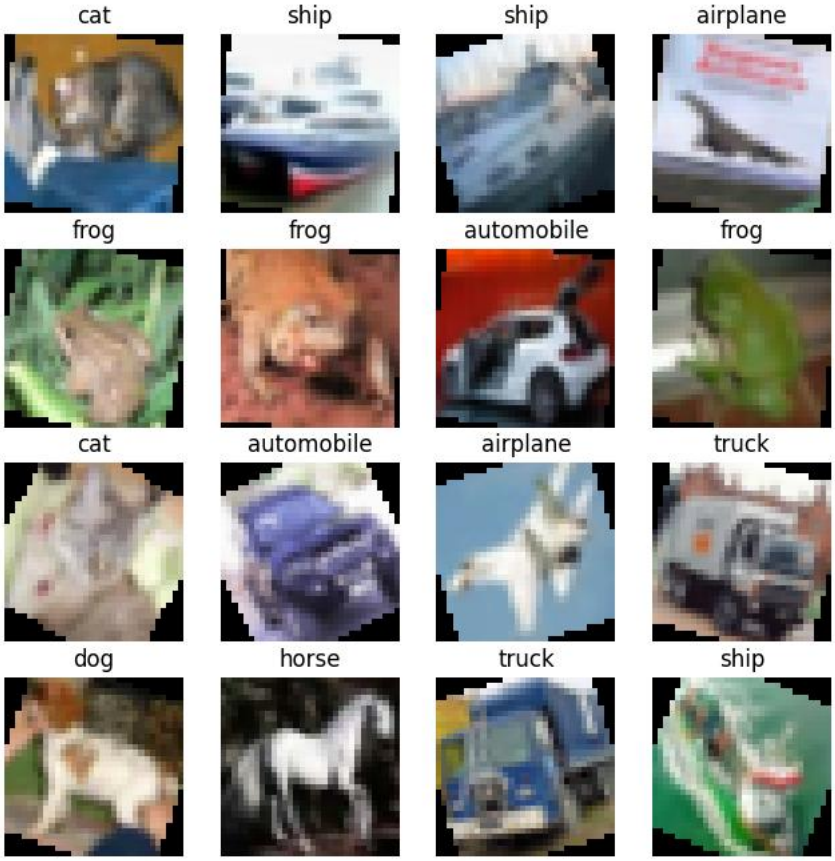
## *1.5 Image data augmentation transformations*

- Include the grids of transformed images: rotation, random erasing, color jitter and five crops.

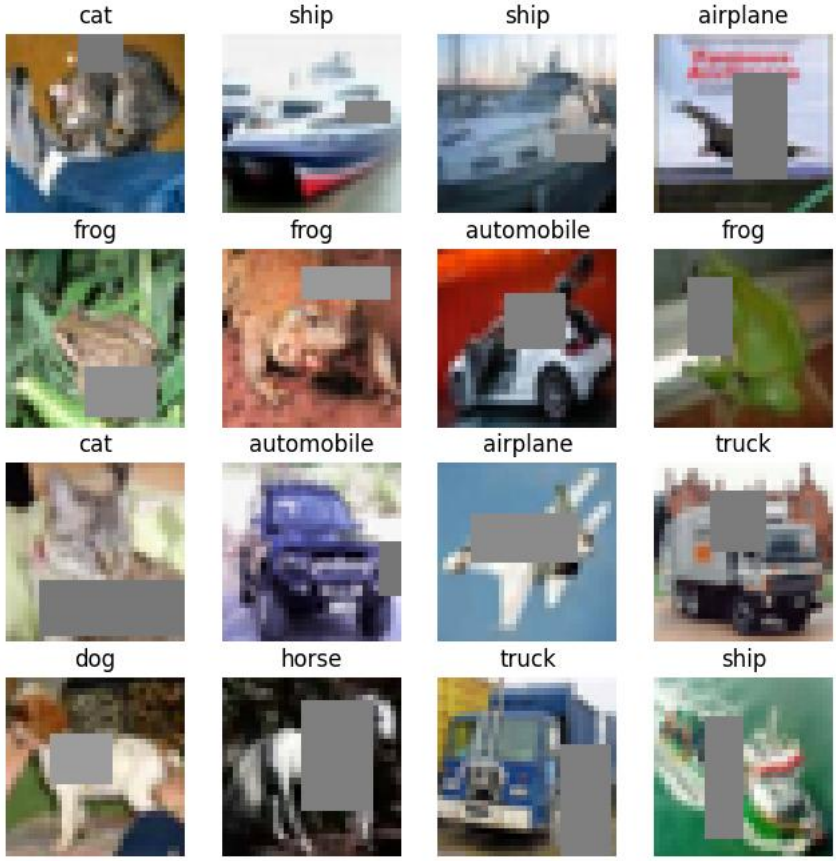
Baseline:



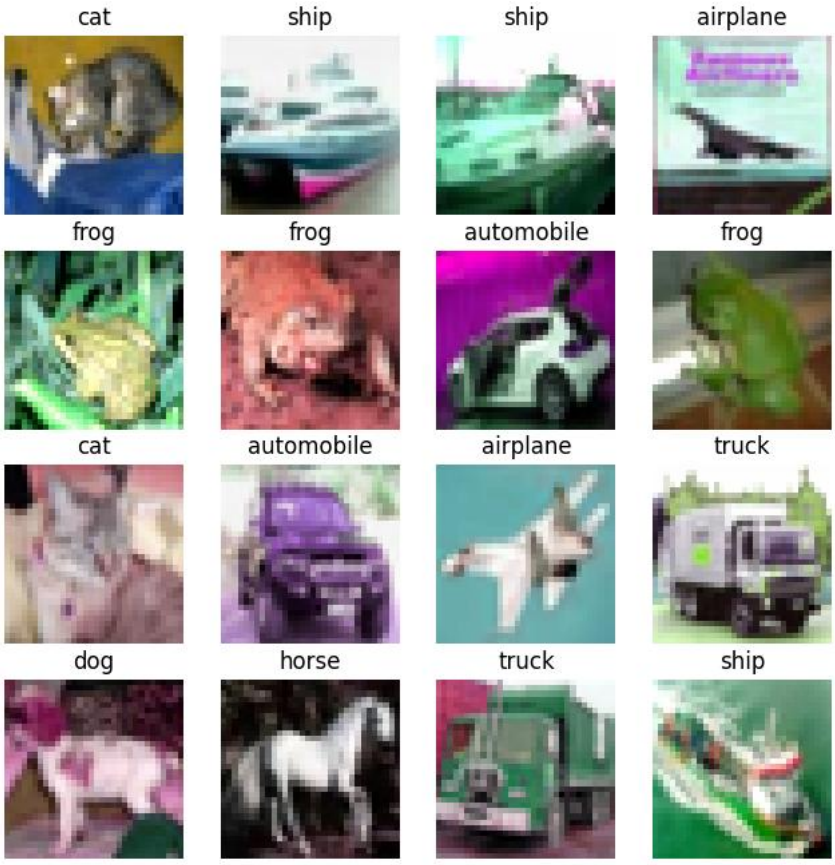

Rotation:



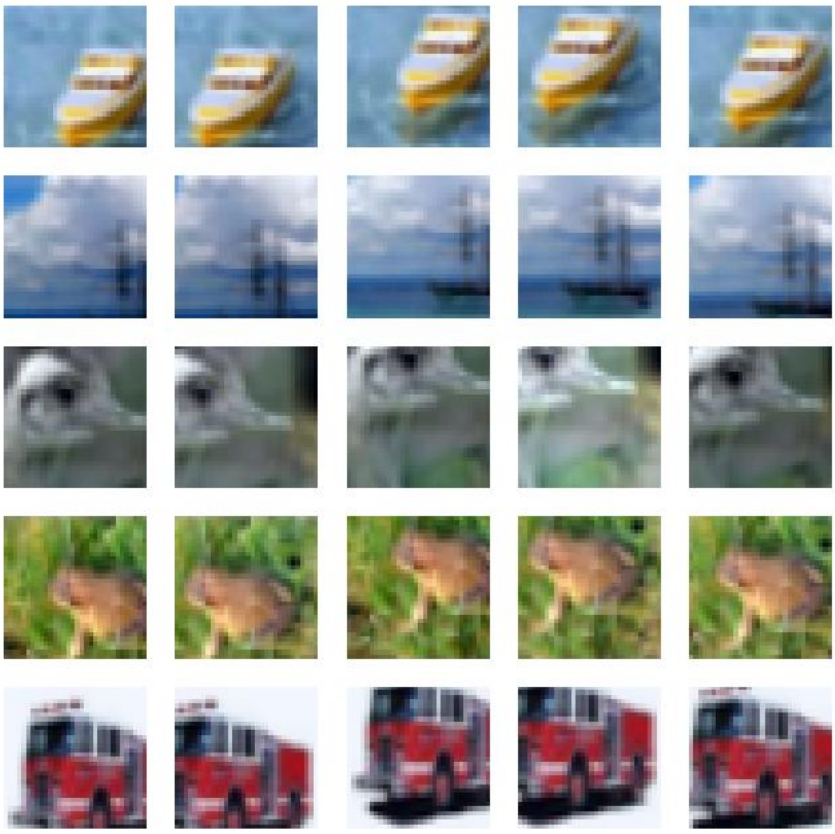

Random eraising:



Color jittering:




FiveCrops:



Each transform gives the model a new challenge to predict the class with an obstacle like in the random rotation or with a new pint of view like with the Five crops color jittering or rotation. However it is very difficult to know which transformation will fit better to each model-dataset.

You can find here the fifth Colab notebook. You must complete the code in the notebook and answer the questions in Section 1.6

## *1.6 Using data augmentation to train a CNN*

- Report the results of your tests with the different data augmentation techniques. Include the transformation/s of each test, any relevant parameter for those transformations, and the training time of the experiment.

| Data augmentation | Relevant parameters for data augmentation (if any) e.g. max. rotation angle | Accuracy | Training time (s) |
|---|---|---|---|
| No augmentation | None | 70% | 480 |
| Horizontal Flip | Rotation angle= 30% | 75% | 600 |
| Random Cropping | Five crops with a resize to 32 and a normalization of: (0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010) | 80% | 900 |
| Erase | p=1, scale=(0.02, 0.25), value=0.5, inplace=False | 76% | 780 |
| Color jittering | brightness = 0.3, contrast= 0.3,saturation = 0.2, hue = 0.2 | 79% | 900 |
| Combination of all the previous transformations | All previous parameters | 82% | 1320 |

The results of the different data augmentation strategies clearly show their positive impact on both model accuracy and training time. Without any augmentation, the model achieves 70% accuracy in 480 seconds. Introducing simple transformations such as horizontal flips with random rotations (30°) increases accuracy to 75%, though training time rises moderately to 600 seconds, indicating improved generalization at a minimal computational cost.

More complex augmentations, such as random cropping and color jittering, further enhance performance to 80% and 79%, respectively. These techniques expose the model to a wider variety of visual contexts—variations in scale, brightness, and color—which make the learned representations more robust. The erase transformation, which randomly removes parts of the image, achieves 76% accuracy, showing that occlusion-based augmentations can also help but are slightly less effective when applied in isolation.

Finally, when all transformations are combined, the model achieves its highest accuracy of 82%, although training time increases to 1320 seconds. This confirms that applying multiple augmentation techniques together improves the model's ability to generalize to unseen data, at the expense of longer training. Overall, the results highlight a clear trade-off between computational cost and performance, with combined augmentations providing the best balance between accuracy and robustness.